

Laboratorio IA 2: Exploracion y explotacion

Materia: Inteligencia Artificial

Dinamica: practica en terminal **Entregable:** experimento y conclusiones breves

Complejizar el robot de la clase 1 para observar una tension central del aprendizaje: explorar soluciones nuevas o explotar la mejor solucion conocida. El robot evoluciona una poblacion de secuencias de movimiento hasta encontrar una ruta desde la esquina superior izquierda hasta la meta.

1. La diferencia en dos minutos

Explorabilidad es la capacidad del sistema para probar regiones nuevas del espacio de soluciones. En el programa aparece como una tasa de mutacion alta: muchos genes cambian y la poblacion mantiene variedad.

Explotabilidad es la capacidad para aprovechar una solucion que ya funciona. En el programa aparece como una tasa de mutacion baja y elitismo: se conservan los mejores individuos y se hacen cambios pequenos alrededor de ellos.

No son dos objetivos completamente separados. Una poblacion sin exploracion puede quedarse atrapada; una poblacion que solo explora puede no consolidar nunca una buena ruta. La presion selectiva debe equilibrar descubrimiento y conservacion.

2. Preparar y ejecutar

Coloca `robot_exploracion_explotacion.py` en la misma carpeta que este manual y ejecuta:

```
python3 robot_exploracion_explotacion.py
```

Para repetir exactamente el experimento, usa una semilla:

```
python3 robot_exploracion_explotacion.py --semilla 7
```

El programa ejecuta dos poblaciones. **EXPLORACION** muta cada gen con probabilidad 0.30; **EXPLOTACION** usa 0.03. Al final muestra la generacion en que cada una encontro la meta y su diversidad final.

3. Como leer el programa

1. `recorrer(adn)` traduce una secuencia de letras U, D, L y R en una trayectoria dentro de una cuadrícula de 8 por 8.
2. `evaluar(adn)` calcula el fitness: premia acercarse a la meta, avanzar y evitar repetir celdas; llegar a la meta recibe una bonificacion grande.

3. `evolucionar(...)` crea 80 robots, conserva los 8 mejores y genera descendientes mutados a partir de los 30 mejores.
4. `diversidad(poblacion)` cuenta cuantos ADN diferentes hay. Es el indicador visible de exploracion.

	Exploracion	Explotacion
Presion	Cambios frecuentes	Conservacion del elite
Mutacion	Alta: 0.30	Baja: 0.03
Diversidad	Suele permanecer alta	Suele caer rapidamente
Riesgo	Ruido y poca estabilidad	Convergencia prematura

4. Experimento guiado

1. Ejecuta el modo comparativo tres veces con semillas 7, 21 y 42.
2. Anota para cada modo las generaciones, el puntaje y la diversidad final.
3. Ejecuta solo cada presion selectiva:

```
python3 robot_exploracion_explotacion.py --modo exploracion --semilla 21
python3 robot_exploracion_explotacion.py --modo explotacion --semilla 21
```

4. Cambia la tasa de mutacion en el diccionario `experimentos` y predice que sucedera antes de ejecutar.

1. Que modo conserva mas ADN diferentes? Por que eso representa exploracion?
2. Una mayor diversidad garantiza llegar antes a la meta?
3. Que ocurre si la mutacion es casi cero? Y si se acerca a uno?
4. El fitness actual premia siempre el camino mas corto? Propone una modificacion y justifica su presion selectiva.

5. Reto de modificacion

Elige una sola modificacion y conserva el resto del programa:

- agrega un obstaculo en una coordenada y penaliza las trayectorias que lo visiten;
- cambia la meta y explica si las dos presiones siguen encontrandola;
- agrega un costo por cada paso para favorecer rutas eficientes;
- implementa una tasa de mutacion que disminuya cuando el puntaje mejore: exploracion al principio y explotacion al final.

6. Entrega

Entrega el archivo Python modificado y una tabla con tres semillas. Escribe entre 5 y 8 lineas respondiendo: *Como cambio la presion selectiva el comportamiento de la poblacion?* No es necesario modificar `robot.py`: esta clase trabaja con la nueva version `robot_exploracion_explotacion.py`.

7. Código completo

El siguiente bloque se inserta directamente desde el archivo Python de la practica. Así, el manual y el programa mantienen siempre la misma version.

```
import argparse
import os
import random
import time

COMANDOS = ("U", "D", "L", "R")
TAMANO = 8
META = (7, 7)
LONGITUD_ADN = 14

def limpiar_pantalla():
    os.system("cls" if os.name == "nt" else "clear")

def recorrer(adn):
    fila, columna = 0, 0
    trayectoria = [(fila, columna)]

    for comando in adn:
        if comando == "U":
            fila = max(0, fila - 1)
        elif comando == "D":
            fila = min(TAMANO - 1, fila + 1)
        elif comando == "L":
            columna = max(0, columna - 1)
        elif comando == "R":
            columna = min(TAMANO - 1, columna + 1)
        trayectoria.append((fila, columna))

    return trayectoria

def evaluar(adn):
    trayectoria = recorrer(adn)
    posicion = trayectoria[-1]
    distancia = abs(META[0] - posicion[0]) + abs(META[1] - posicion[1])
    pasos utiles = sum(
        actual != anterior
        for anterior, actual in zip(trayectoria, trayectoria[1:])
    )
    visitas_repetidas = len(trayectoria) - len(set(trayectoria))

    puntaje = 1000 - distancia * 40
    puntaje += pasos utiles * 2
    puntaje -= visitas_repetidas
    if posicion == META:
        puntaje += 1000
    return puntaje

def diversidad(poblacion):
    return len({" ".join(adn) for adn in poblacion})

def mutar(adn, tasa_mutacion, rng):
    return [
        rng.choice(COMANDOS) if rng.random() < tasa_mutacion else comando
        for comando in adn
    ]
```

```

]

def evolucionar(nombre, tasa_mutacion, semilla, maximo_generaciones=2500):
    rng = random.Random(semilla)
    poblacion = [
        [rng.choice(COMANDOS) for _ in range(LONGITUD_ADN)]
        for _ in range(80)
    ]
    historial = []

    for generacion in range(maximo_generaciones + 1):
        poblacion.sort(key=evaluar, reverse=True)
        mejor = poblacion[0]
        puntaje = evaluar(mejor)
        historial.append((generacion, puntaje, diversidad(poblacion)))

        if recorrer(mejor)[-1] == META:
            return {
                "nombre": nombre,
                "adn": mejor,
                "generacion": generacion,
                "puntaje": puntaje,
                "diversidad": diversidad(poblacion),
                "historial": historial,
            }

        elite = poblacion[:8]
        nueva_poblacion = [robot[:] for robot in elite]
        while len(nueva_poblacion) < len(poblacion):
            padre = rng.choice(poblacion[:30])
            nueva_poblacion.append(mutar(padre, tasa_mutacion, rng))
        poblacion = nueva_poblacion

    return {
        "nombre": nombre,
        "adn": poblacion[0],
        "generacion": maximo_generaciones,
        "puntaje": evaluar(poblacion[0]),
        "diversidad": diversidad(poblacion),
        "historial": historial,
    }

def dibujar(adn, titulo, generacion, puntaje, pausa):
    trayectoria = recorrer(adn)
    for paso, posicion in enumerate(trayectoria[1:], start=1):
        limpiar_pantalla()
        print(f"{titulo} | Gen {generacion} | Puntos: {puntaje}")
        print("Exploracion: muta; explotacion: conserva.\n")
        for fila in range(TAMANO):
            linea = ""
            for columna in range(TAMANO):
                celda = (fila, columna)
                if celda == META:
                    linea += " " if posicion == META else " "
                elif celda == posicion:
                    linea += " "
                else:
                    linea += " "
            print(linea)
        print(f"Paso {paso}/{len(adn)} | ADN: {''.join(adn)}")
        if pausa:
            time.sleep(pausa)

```

```

def mostrar_resultado(resultado, pausa):
    dibujar(
        resultado["adn"],
        resultado["nombre"],
        resultado["generacion"],
        resultado["puntaje"],
        0 if pausa == 0 else 0.08,
    )
    print(f"Diversidad final: {resultado['diversidad']}/80")
    if pausa:
        time.sleep(1.5)

def main():
    parser = argparse.ArgumentParser(
        description="Compara exploracion y explotacion en una poblacion de robots."
    )
    parser.add_argument(
        "--modo",
        choices=("comparar", "exploracion", "explotacion"),
        default="comparar",
    )
    parser.add_argument("--semilla", type=int, default=7)
    parser.add_argument("--sin-pausa", action="store_true")
    args = parser.parse_args()
    pausa = 0 if args.sin_pausa else 1.5

    experimentos = {
        "exploracion": ("EXPLORACION", 0.30),
        "explotacion": ("EXPLOTACION", 0.03),
    }
    modos = (args.modo,) if args.modo != "comparar" else experimentos
    resultados = []

    for indice, modo in enumerate(modos):
        nombre, tasa = experimentos[modo]
        resultado = evolucionar(nombre, tasa, args.semilla + indice)
        resultados.append(resultado)
        mostrar_resultado(resultado, pausa)

    if len(resultados) == 2:
        print("\nCOMPARACION")
        print("Modo          Mutacion  Generaciones  Diversidad final")
        print("-----")
        for resultado in resultados:
            modo = resultado["nombre"].lower()
            tasa = experimentos[modo][1]
            print(
                f"{modo:<14}{tasa:<10.2f}{resultado['generacion']:<14}"
                f"{resultado['diversidad']}/80"
            )

if __name__ == "__main__":
    main()

```